CUSTOM ORGANIZATION MANAGEMENT SYSTEM

COMS on Workday Extend

Delegating shell roles safely: built for every application, now checked against Workday itself

Off Appian, inside Workday: application teams create their own shell organizations, and see only their own roles.

XTND-0037-XFD_SEC-Custom_Organization_Management · Crispin Mwanyumba · Integrated Service Centre · 2026-11-07

OVERVIEW 1/3 · SHELL ROLES

A three-legged stool, with keys for other buildings

Role assignee

Usually a position: the desk, not the person

Role assigner

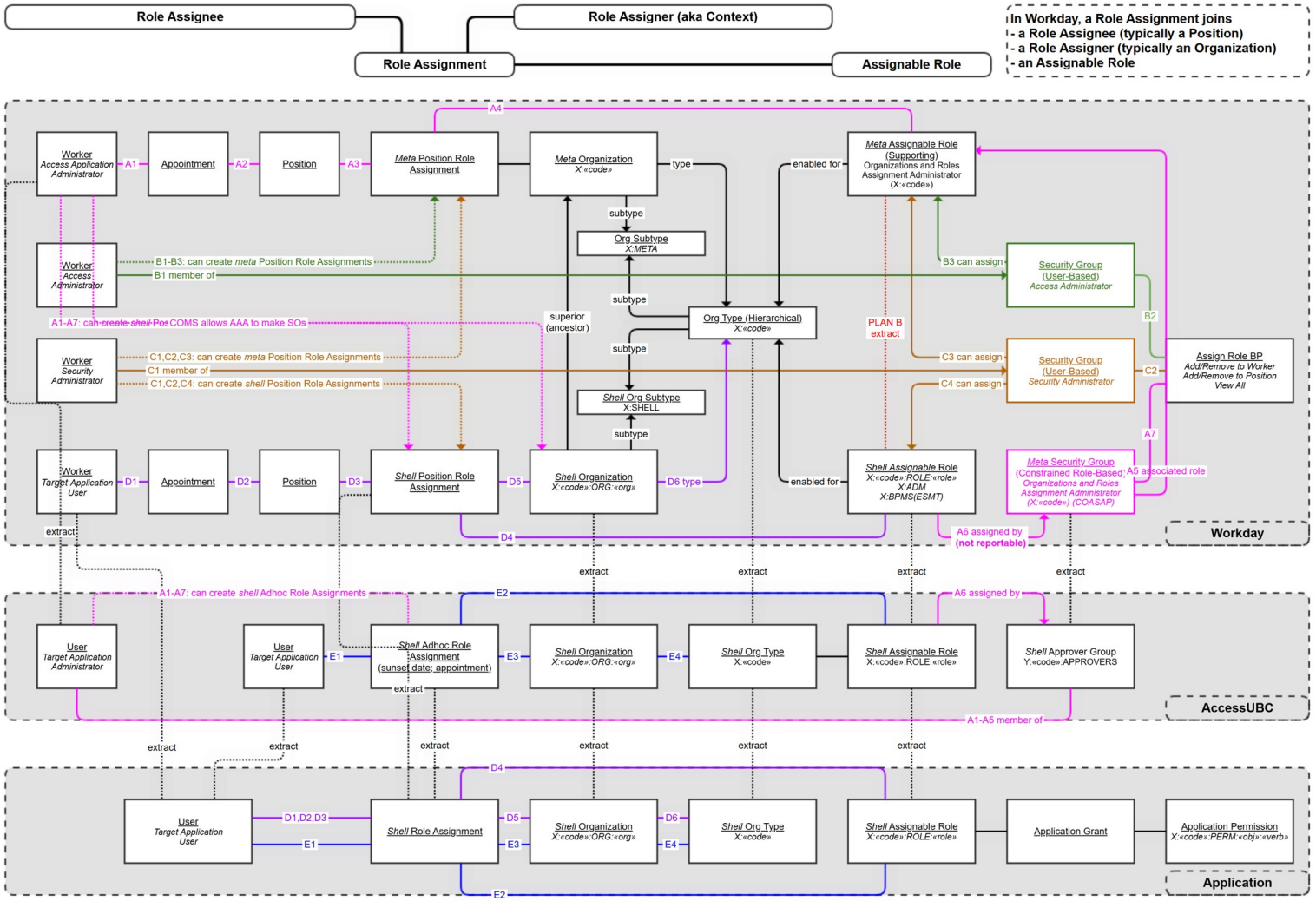
The context: a shell organization such as X:CLW:ORG:…

Assignable role

A shell role such as X:CLW:ROLE:…

“Shell” = no security group attached, so no power inside Workday. The keys only open doors in other buildings: AccessUBC, Clockwork, OAMS, next AWS.

Must-have: leave the position, lose the access. Nice-to-have: the replacement gets it automatically.



OVERVIEW 2/3 · TODAY

Three tiers, joined by extracts

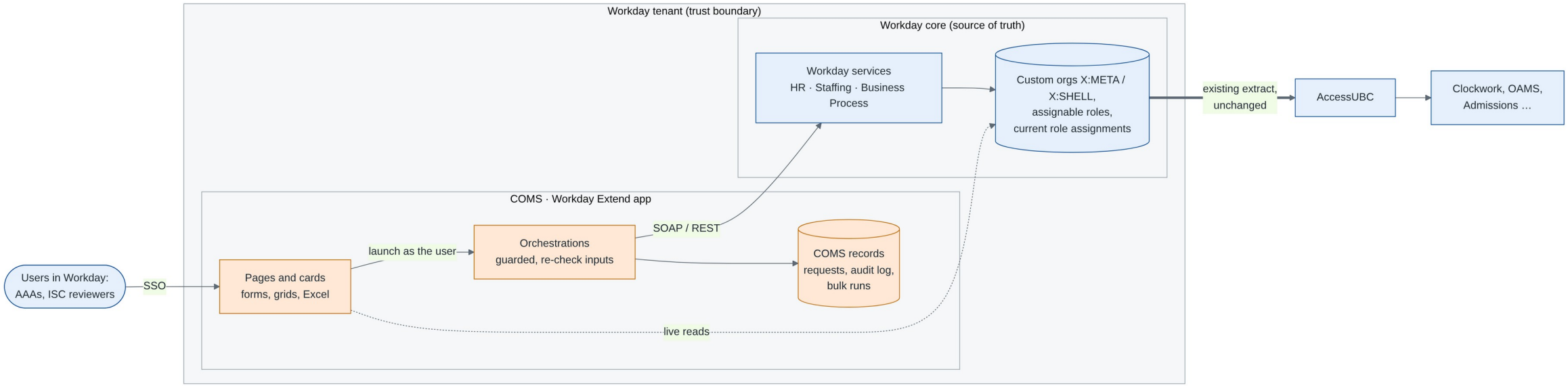
Workday holds the truth: positions, shell orgs, shell roles, role assignments.

AccessUBC adds ad hoc assignments and approver groups.

The application turns roles into grants and permissions.

COMS · Current architecture diagram from the solution architect

Where COMS sits: inside Workday, upstream of AccessUBC



Inside
the Workday tenant: pages, logic and app data

3
Workday services it calls: HR, Staffing, Business Process

1
boundary crossing: the existing AccessUBC extract

0
changes to AccessUBC or the target applications

THE BRIEF, PLAYED BACK

Delegate shell organization management without putting Workday at risk

OBJECTIVE 1

Exit Appian

Move COMS into Workday Extend, so the Appian license can go.

OBJECTIVE 2

Learn Extend safely

A first Extend app with few users, low change management and SMEs close by.

THE POINT

Fewer ISC tickets

Clockwork-style teams create shell orgs themselves, inside their own boundary.

And rebuild it “the right way”: one schema-driven tool any downstream application can use, not a Clockwork-only form.

WHAT HURTS TODAY

Three pains the rebuild has to fix

ISC is the bottleneck

Only ISC can create shell orgs and assignable roles. Clockwork needs new orgs every term: a ticket each.

Built too narrow

The Appian COMS handles one pattern shape. Each new tool would need a rebuild.

AAA sees too much

A AAA in Workday can see others' role assignments and request their deletion: two errors from a loss.

WHERE WE ARE

Phase A is built for every application; B and C are well started

Phase	The architect's ask	Built in dev today	Still to do
A	Off Appian: delegated shell orgs, bounded by org type	Shell orgs with X:META / X:SHELL naming; one pattern form for 15 apps, formats kept in the app; inactivate, reactivate; audit	Lock records (ISU), Clockwork pilot
B	Manage shell assignable roles without ISC tickets	Role and AAA request forms, ISC review with reminders, waiting-time tracking, Home cards	Delegate bounded roles
C	Shell role assignments, seeing only your own	One by one, grid or Excel, browsing by supervisory org; Access Review; X: shell roles only	Scope each user to their apps

Every change is checked before it reaches Workday and recorded after it.

THIS WEEK

What changed on 6 and 7 November

PICKED FROM WORKDAY

Academic unit, course subject and academic period are chosen from Workday lists, not typed

CHECKED TWICE, AT SCALE

Each value is looked up on its own, on the page and on the server: quick with tens of thousands of rows

RULES OR REGEX

A field’s format is a simple rule or a regular expression; switch between them, copy either way

THE VALUE, MAPPED

Value stream map: today’s ticket and queue against a checked change in seconds

TEAM NAMING STANDARD

App and orchestrations renamed to the XTND / IORC standard; Workday links kept

MORE GATES ON DEPLOY

Workday’s own App Builder validation and a code review run on every deploy

Also: the checks are switched on for Clockwork (Subject, Academic Period) and Final Grades (Subject); tests read live data, so they keep working when Workday data changes.

PHASE A IN ONE STORY

A Clockwork admin sets up next term, no ticket

1 · CHOOSE

Application Clockwork; academic unit, subject, course, section, term

2 · PREVIEW

COMS builds the name and checks each part against Clockwork's format

3 · CREATE

A custom org of type X:CLW, subtype X:SHELL, in Workday

4 · USE

Assign positions; the next extract carries it to Clockwork

Example of a pattern organization created in dev

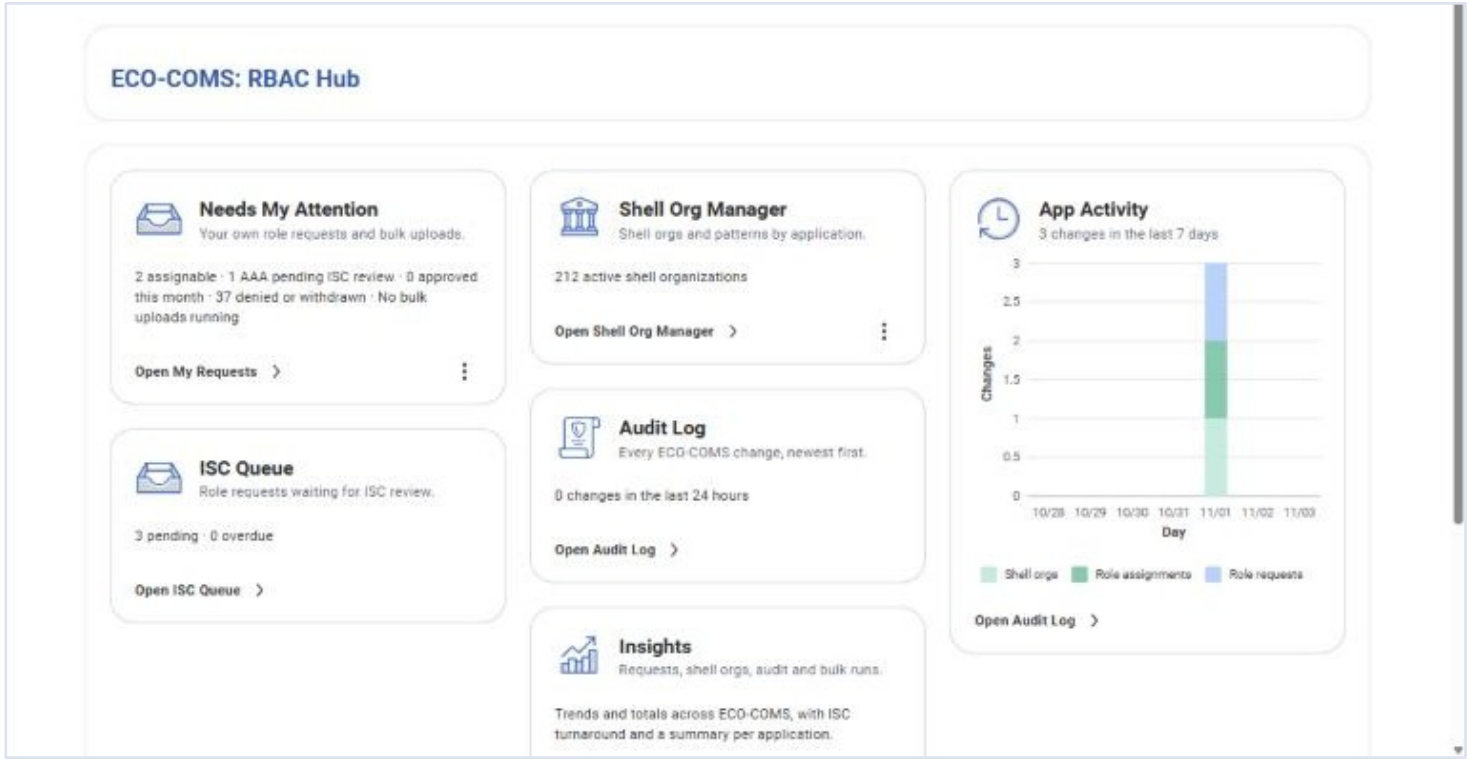
X:CLW:PATTERN:MATH:MATH_V:303:951:2026W1

One front door

The RBAC Hub answers four questions: what needs me, what ISC still has to review, what changed, and how busy COMS has been.

App Activity: changes per day for a week. New since: Pending Requests by Age, and an Access Review card for vacant positions.

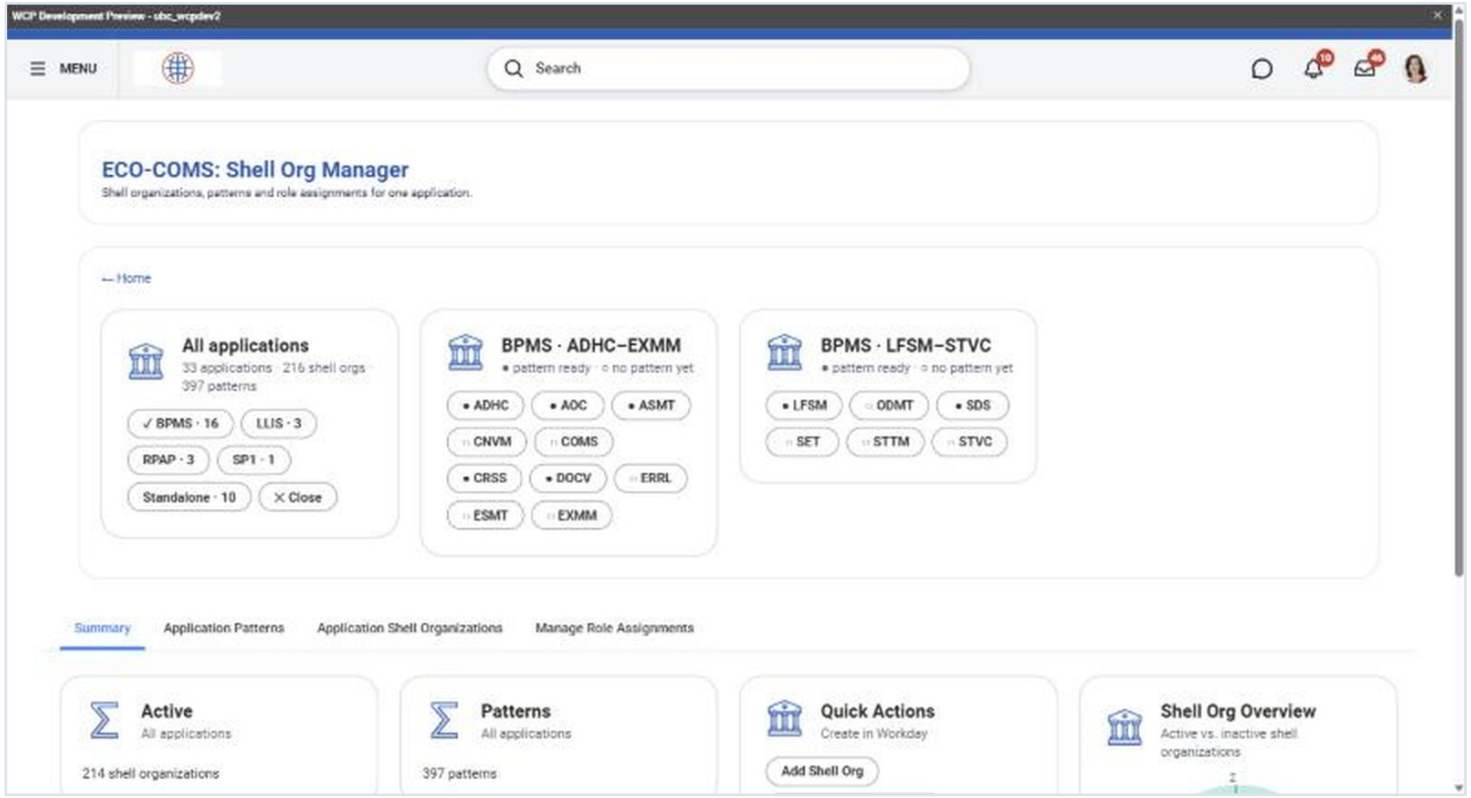
Every card is a doorway, with its own icon used everywhere it appears.



The map of every key ring

Shell Org Manager: every application, its patterns and organizations, and who holds which role.

All 33 applications, found from the organizations themselves. Pick a family, then an application, and the whole page narrows to it.



PRODUCT TOUR · 3

Inside Workday, where people already work

Workday Home cards: “My Role Requests · 3 pending” and “ISC Queue · 3 waiting”, with the oldest item and a link straight to it.

A COMS section on the Worker Profile, review steps in the Workday inbox, and one look across every page.

No new place to remember: COMS comes to the user.

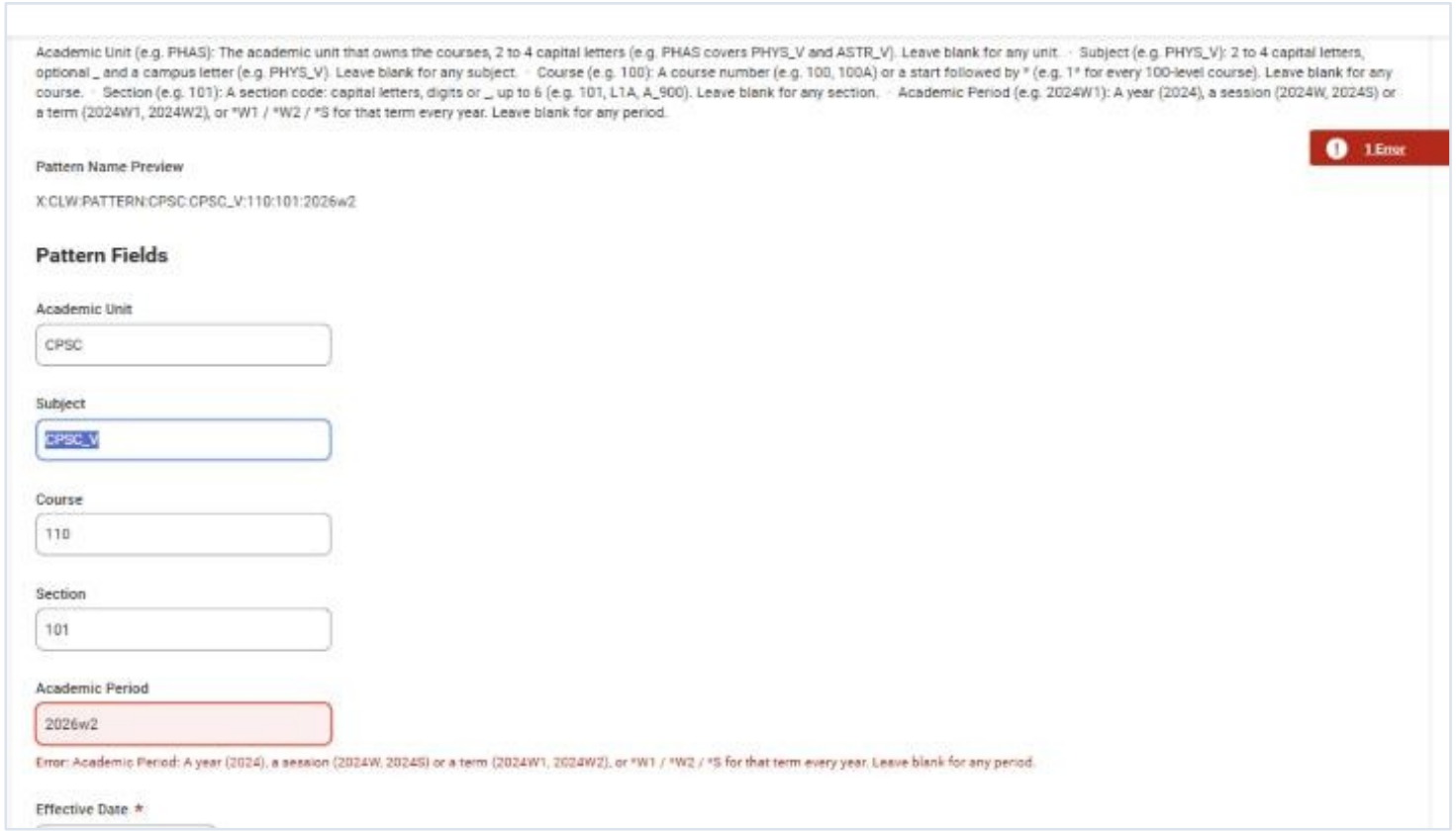


One form, every application's rules

The form reads the application's format: field names, examples and rules come from its schema record, maintained in the app.

The name builds live; a field that breaks the rule says what it expects. Weekend dates and existing patterns are refused before Workday.

Then a receipt, with a link to its audit record.

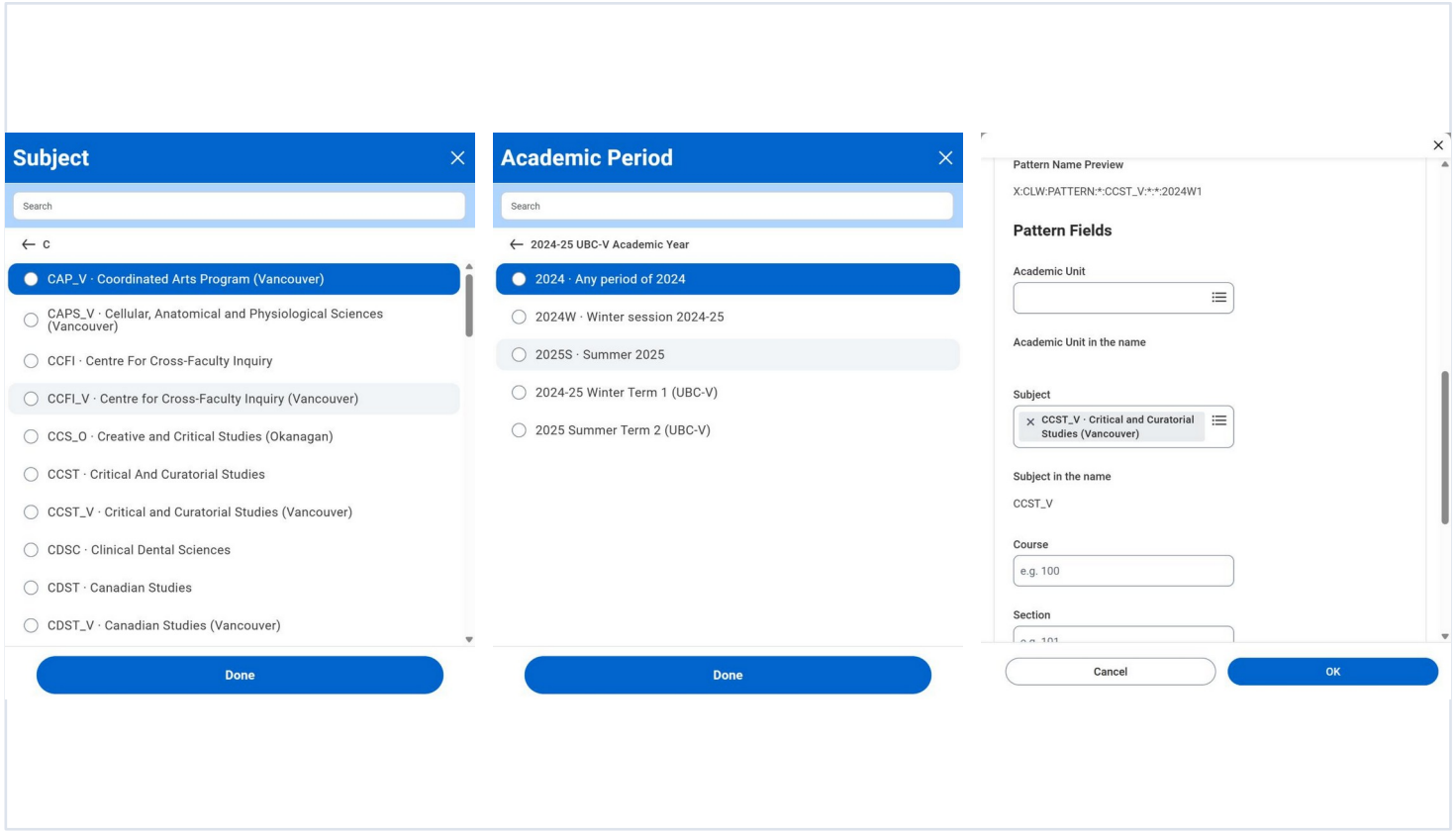


Picked from Workday, not typed

Academic unit, course subject and academic period come from Workday: browse (units by subtype, subjects A–Z, periods by academic year) or type a code or name to search.

The box below shows what goes into the name: a period becomes its short form (2024-25 Winter Term 1 → 2024W1); wildcards and sessions (*, *W1, 2024W, 2025S) are choices too.

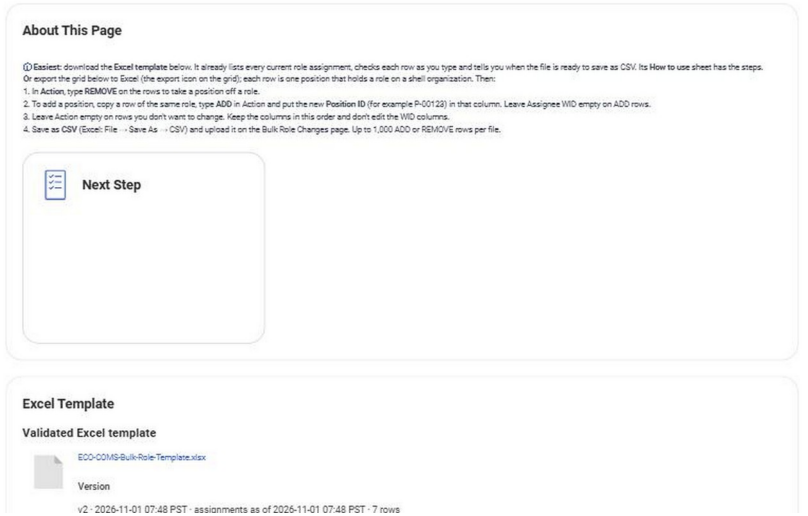
Each value is one small Workday query, checked on the page and again on the server, so it stays quick with tens of thousands of subjects or periods.



PRODUCT TOUR · 6

Bulk changes, three ways

Way	Best for	Size
Manage one role	A quick add or remove	A few positions
In-app grid	Edit live data, like an EIB	Up to 50 per submit
Excel template	Large or offline changes	Up to 1,000 per file



Every bulk run can be a dress rehearsal first: validate checks each row in Workday and changes nothing, then Apply makes it real.

PRODUCT TOUR · 7

Trust is built in, not bolted on

No self-approval

Workday excludes the requester from approving their own request.

Every change recorded

Audit log of every create, status and role change, linked to its Workday event.

Success means success

The app re-reads Workday before it says “done”; a rejection is shown, never hidden.

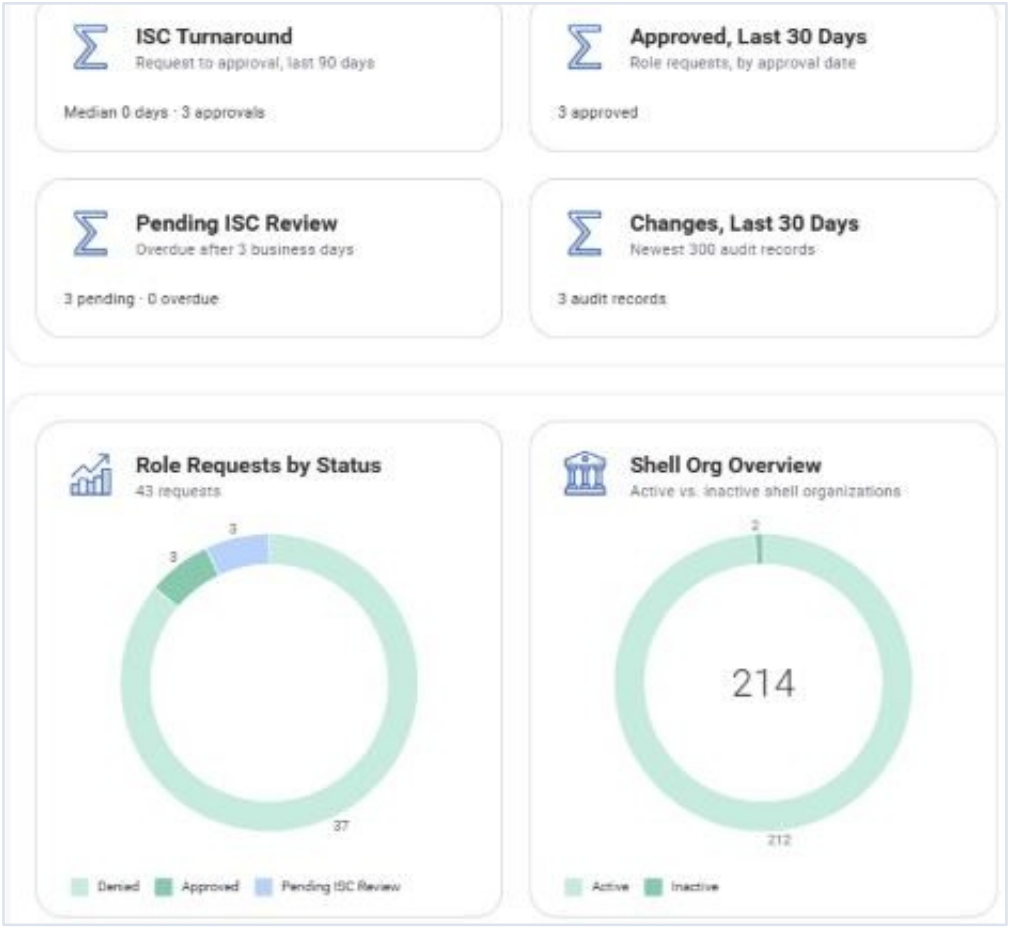
You see only your part

Seven Workday security domains decide who sees and changes what.

Insights: is it working, and for whom?

One page across requests, shell orgs, audit and bulk runs: ISC turnaround, what's pending and overdue, approvals, and a row per application.

For custom reports: four lean Workday report data sources (role requests, audit trail, bulk runs, applications).



ARCHITECTURE AS BUILT (PHASE A)

Four layers, all inside the Workday tenant

PEOPLE

Requesters · ISC reviewers · Admins, in the Workday web and mobile UI

PAGES

40 pages and 45 cards, plus 2 Workday Home cards; forms check input before submit

LOGIC

13 orchestrations + 4 shared sub-orchestrations: orgs, status, roles, reviews, bulk runs, Home cards

APP DATA

5 business objects + 2 file stores, 15 application schemas, 4 report data sources

LIVE READS

6 WQL queries and 1 Graph query: orgs, roles, positions and audit, straight from Workday

WORKDAY CORE

Custom Organizations · Assign Roles · Review business process · Security domains

Like a restaurant: pages are the dining room, orchestrations the kitchen, business objects the pantry, Workday the farm.

PROPOSED TARGET ARCHITECTURE

COMS as the delegated control plane for shell roles

① Context
My apps, from my AAA meta assignments

② Schema registry
Per app: prefix, type, columns, rules

③ Pages
One generic form, grid, Excel

④ Guarded orchestrations
Re-check context and schema

⑤ Audit
Written by an integration user

⑥ COMS API
For automation, e.g. AWS

WORKDAY CORE (source of truth)
Custom org hierarchy X:«code» / X:SHELL · shell assignable roles · position role assignments · review process

ACCESSUBC
Extracted shell orgs, roles and assignments

APPS
Clockwork, OAMS, … AWS: roles to grants

WORKDAY TENANT

COMS (Workday Extend)

Orange = new in the proposal · grey = built and reused · AccessUBC and the apps are unchanged.

PROPOSED DATA FLOW

Access follows the position, both ways

1 Sign in
Context = my apps

2 Enter
Form, grid or Excel

3 Check
Schema + context, twice

4 Write
Workday org or role API

5 Record
Audit, then receipt

6 Extract
AccessUBC → app

Worker leaves position
Termination or move

Assignment stays on desk
Nobody in the seat

Next extract
Access turns off

Replacement hired
Access turns on again

PROVISION · driven by the application admin

DEPROVISION · driven by HR, no COMS step

PROPOSAL · CONTEXT-SCOPED ACCESS (PHASE C)

If you can’t see it, you can’t make the first error

Workday task today

AAA sees every role on a position

Including real Workday roles

Can request deletion of others’

COMS, scoped

Sees only X:«my code» shell roles

Adds and removes only those

Server re-checks every change

How: COMS reads the user’s own AAA meta assignments (role on meta org X:«code») and filters every page, search and orchestration by those codes. Admins of two apps pick which one to manage.

BUILT · SCHEMA-DRIVEN (THE “RIGHT WAY”)

A new application is a new record, not a new release

Schema field	Clockwork (live)	Final Grades Submission (live)
Prefix and type	X:CLW:PATTERN · subtype X:SHELL	X:BPMS(AOC):CP · subtype X:SHELL
Columns	Academic unit, subject, course, section, academic period	Subject, course
Rules per column	A rule per column (type, length, characters) or a regular expression; switch and copy either way	Same: subject is two to four capital letters, optional campus letter
Blank / wildcard	Blank means any (*), coarse to granular	Same engine
Lookups	Academic unit, subject and period checked in Workday and picked from lists	Subject checked in Workday and picked from a list

15 applications have a schema today, maintained in the app on Maintain Application Pattern. One engine reads it to draw the form, build the name, check on the page and re-check on the server.

HOW IT WORKS · ONE CHANGE

Every change is sent like registered mail

Page checks
Rules, dates, duplicates

Guard
Orchestration re-checks every input

Workday call
Web service, e.g. Assign Roles

Audit
Written after success, linked to the Workday event

Receipt
Page confirms it really happened

If Workday says no, the user sees Workday's reason on the page, and nothing is logged as done.

HOW IT WORKS · BULK ENGINE

One engine behind the grid and the file

Limit	Value	Why
Grid submit	50 changes	A page request must finish within 60 seconds
Excel / CSV file	1,000 changes	A background run in production must finish within 60 minutes

Two front doors

The grid writes its rows as drafts; the file upload stores the CSV. Same row format, same checks.

Runs in the background

An asynchronous orchestration takes each row: look up the position, call Assign Roles, record the result.

Keeps going

A rejected row is marked FAILED with Workday's reason; the rest still run. Results refresh live.

Locks on every door, not just the front one

Layer	Control	Status
Pages	7 security domains decide who sees each page	Live
Orchestrations	Each user-launched one is secured by its page's domain	Live
Inputs	Guards re-check every parameter before Workday is called	Live
Approvals	Business process with Exclude Initiator	Live
Records	Audit and request records written only by an integration user	Built; awaiting ISU setup
Context	Each user sees and changes only their own apps' shell roles	Proposed (Phase C)

Like a hotel: your key card opens your floor, the safe has its own code, and the front desk keeps the log.

HOW IT WORKS · ENGINEERING

Every change goes through inspection before the road

Commit

Code in GitHub; pages built by scripts

Static checks

Known Workday pitfalls, App Builder validation and a code review block the upload

Deploy

App Hub upload, tenant deploy; build errors shown in the run

Live tests

Smoke tests against the real tenant

513 commits since Oct 29, 2026 · 32 test suites, static to security · 1–3 min from push to deployed

AT A GLANCE

What we built

40
pages

52
cards, plus 2 on Workday Home

13 + 4
orchestrations + sub-orchestrations

5 + 2
business objects + file stores

15
application schemas

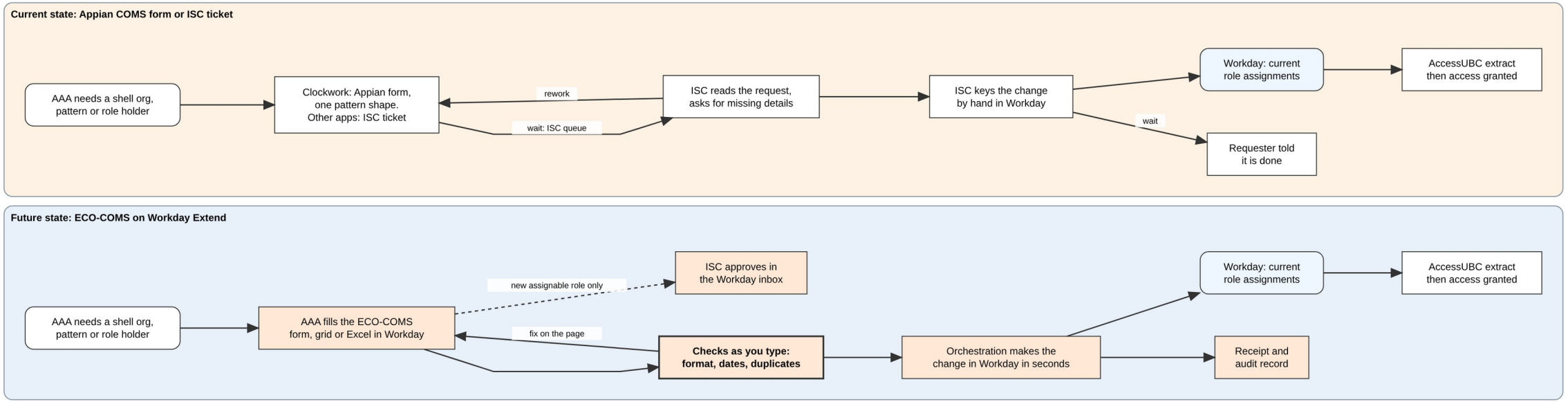
7
security domains

1
review business process

0
servers outside Workday

THE VALUE, BEFORE AND AFTER

From a ticket in a queue to a checked change in seconds



4 → 2
steps from need to change in Workday

2 → 0
hand-offs for a routine change

Queue → 0
waiting for ISC on a routine change

2 → 1
platforms to run: Appian retired

COMS · Value stream map, also in the Business Requirements page (section 4) · ISC queue time to be measured

WHY EXTEND, NOT APPIAN

The value lives where the data already is

1 · ONE PLATFORM

Runs inside the Workday tenant: the same sign-in, security and business processes. No Appian licence, hosting or second support team.

2 · EVERY APPLICATION

A new application’s naming format is a record, not a rebuild. Appian handles one pattern shape; COMS has 15 schemas today.

3 · SAFE BY DESIGN

Checked as you type and again on the server, a Workday approval with no self-approval, and an audit log linked to each Workday event.

4 · SAME PIPES

AccessUBC still extracts the current role assignments from Workday. The target applications get the same data, sooner.

Appian’s cost goes; the delegation, the checks and the evidence move into Workday, next to the data they protect.

DELIVERY PLAN

Retire Appian first, then widen the delegation

1 · FINISH A

ISU-written records, Clockwork pilot, promote, cut over and retire Appian COMS

2 · SCOPE

Context scoping: each admin sees only their apps (the schema engine is done)

3 · PHASE B

Delegated shell roles inside each app's boundary

4 · PHASE C+

Scoped role assignments replace the AAA task; COMS API for AWS

Same pipeline at every step: static checks, App Hub, deploy, live tests. The integration user is re-assigned in each tenant.

DECISIONS WE NEED

Five questions for this group

#	Question	Our recommendation
1	Scope COMS users by their AAA meta assignments?	Yes: it removes the first error
2	Delegate shell assignable roles inside an app's boundary?	Yes, for X:«code» roles; keep ISC for others
3	Shell orgs: add and inactivate only, never rename?	Yes: an assignment's meaning never changes
4	Has Workday improved custom org security?	Check the latest release before Phase C
5	Expose a COMS API for automation such as AWS?	Yes, after scoping is live

THE ASK

Same pipes downstream, a safer front door upstream

- Agree Phase A is ready for a Clockwork pilot
- Decide on context scoping and Phase B delegation
- Set up the integration system user, then set dates

Questions?